COMP3011J MC Project (Alpha)

22207232 Qiyue Zhu

November 4, 2024

1 Brief Overview of the Open-Source App: Kandean

With the original version of **Kandean**, users can effortlessly create, post, and share their unique recipes, meanwhile, comment, likes, and save various recipes.
The coming version of this app will also provide a digital fridge that tracks available ingredients. The possible coming features are: timely reminders for items nearing expiration, setting automatic updates for the quantity of each item stored and receive shopping suggestions when ingredients run low.
While most of the popular recipe applications focus on creating and sharing recipes or diet planning, **Kandean** uniquely integrates these functionalities with a brand-new digital fridge feature. Having this, you can easily manage your diet and shopping choices.

2 Newly Implemented Features

- **Digital Fridge Display:** the items in the fridge will be displayed in a scrollable page with a search bar above.
- **A Floating Button:** users can add new item by clicking a floating button on the Digital Fridge page, which provide 2 ways to achieve: write the item and scan the barcode of the item (still developing).
- **Detail Storage View:** allow users to view detailed information about storage items in their digital fridge. Highlight that this page displays item details like name, quantity, unit, category, expiration date, and notes, making it easy for users to manage their stored ingredients.
- **Create Item Functionality:** allow users to add more items to their digital fridge.
- **Edit Item Functionality:** allow users to navigate from the Detail Storage View to an Edit page, where they can update item details.

3 Example Explanation of Implementation: Digital Fridge page 1

- **XML Layouts:** *fragment_digital_fridge.xml*, *list_data_storage_all.xml*, *list_storage_search.xml*, *template_list_item_search.xml*
  use various layout to set the page, like the NestedScrollView and LinearLayouts that make the interface intuitive and easy to navigate. Structure each attribute to a frame to separate clearly. Have a different display for the empty page.
- **Adapter Code:** *AllStorageAdapter.java*
  handle the case when the user click a item (the 'onClick()' method) and transfer the data to the item's detail page (pass data to 'DetailStorageFragment' using 'Bundle' to demonstrate data flow).
- **Fragment Code:** *DigitalFridgeFragment.java*
  SearchView for user input to filter the displayed items. SwipeRefreshLayout to handle the pull-to-refresh action for the item list. Floating Action Buttons (FABs) for scanning new items or adding them through handwriting. The getAllStorage() method fetches storage items from a remote API using Retrofit. It displays a shimmering effect while loading data and updates the UI based on whether the fetched data is empty or not. Successful data retrieval populates the RecyclerView with an AllStorageAdapter. The checkConnection() method ensures there's an active internet connection before attempting to load data.

Figure 1: Digital fridge page and the item's details

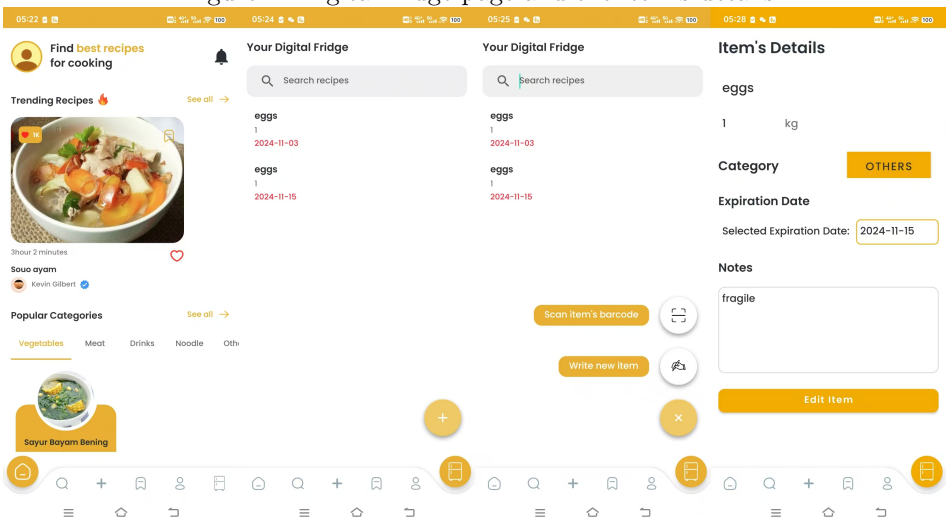


Figure 2: Create and edit an item

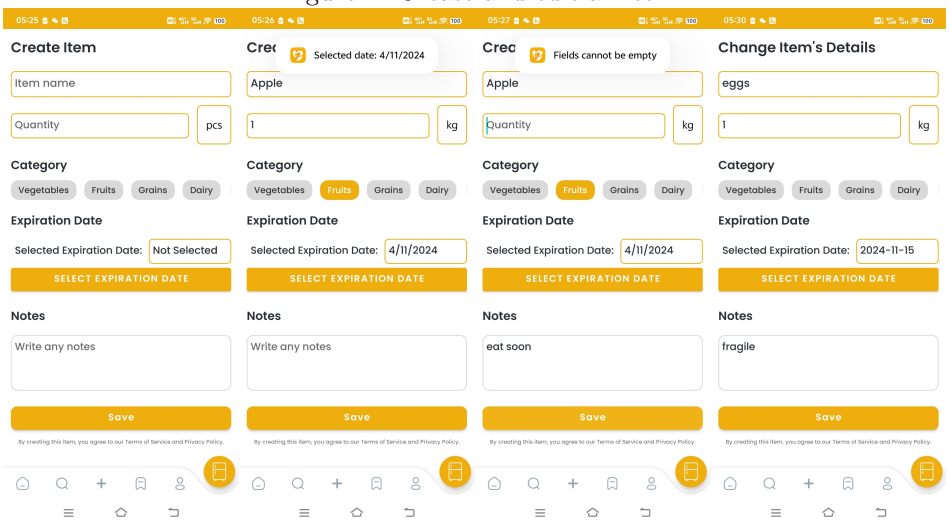


Figure 3: Original Login and register pages

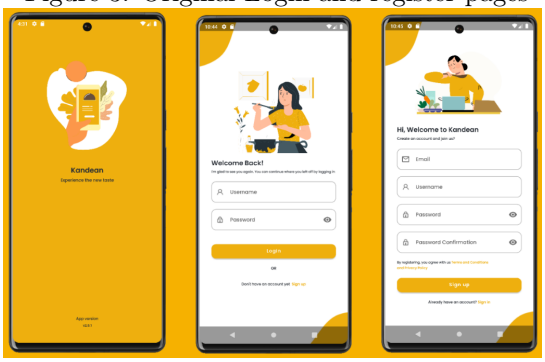
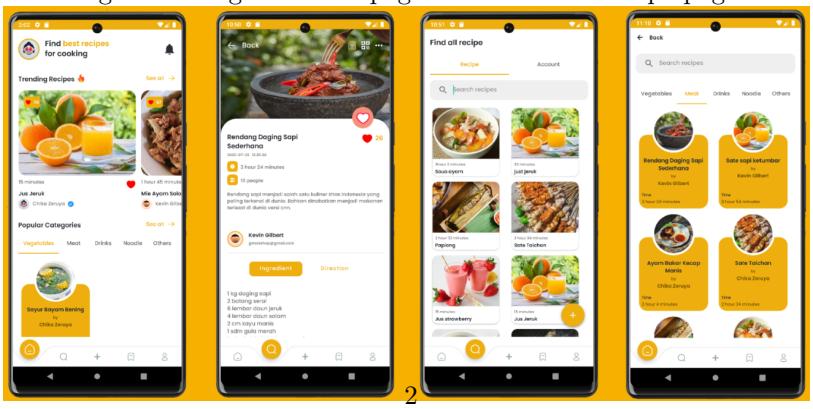


Figure 4: Original Home pages and the basic recipe pages



- **Model Code:** *StorageModel.java*
  The structure of StorageModel.java defines what a storage item looks like, encapsulating the properties and behaviors associated with each stored item. The use of the Serializable interface allows instances of this class to be easily passed between Android components, such as activities and fragments.

- **Interface Code:** *InterfaceStorage.java*
  This interface defines the API endpoints and the expected HTTP methods needed to manage storage items in your digital fridge application using Retrofit. By outlining methods such as getAllStorage, createItem, and deleteStorage, InterfaceStorage.java connects directly with the back-end services via the PHP scripts. The Retrofit library simplifies the interaction by allowing these API calls to easily convert JSON responses from the server into StorageModel objects. This creates a smooth and efficient pathway for the application to request and handle storage item data.

- **php File:** *AllStorageAdapter.java*
  The PHP script serves as a backend service that processes API calls made through InterfaceStorage.java. It interacts with a MySQL database to query and retrieve storage items based on specified user criteria, such as user ID and category. When a request is made, the script constructs the corresponding SQL queries and executes them, returning the resulting data encoded in JSON format. This response is then parsed by Retrofit into instances of StorageModel.java. This seamless integration ensures that the application receives the necessary storage data in a structured format, enabling effective display and management of storage items in the user interface.

4 Development Timeline: Table 1

Time	Work to be completed	Comments
Week 2	Set up the new basic pages and layout of the digital fridge page on the main page	Basic new function setup based on the existing app
Week 4	application's practicality and the user's stickiness based on users' requirements	Add APIs
Week 7	Set up the push notifications, set up the expiration-date-reminder related pages and items	Add APIs
Week 9	Fine-tune the UI and test the functions, try to add more functions if enough time is left	Test and fine-tune

Table 1: Timeline for implementation